SYSTEM ARCHITECTURE HIGH CONCURRENCY PRODUCTION CHECKLIST TURF & FACILITY BOOKING

# Engineering Blueprint: Slot Booking Engine

A Comprehensive Guide to Race Conditions, Expiry Traps, Timezone Edge Cases & Production Bugs

| DOMAIN | CRITICALITY | CONCURRENCY MODEL | TARGET STACK |
|---|---|---|---|
| Facility & Turf Booking | Financial & Inventory Integrity | Distributed Atomic State | Node.js / SQL / Webhooks |

**Executive Architectural Summary:**
A booking slot system seems straightforward at first glance—create time intervals and let users reserve them. In production, however, it represents one of the most volatile system designs in software engineering because it sits at the intersection of **distributed concurrency, payment gateway asynchronous states, physical inventory locks, and international timezones**. Below is the complete catalog of critical bugs that must be guarded against when designing or re-building a slot booking platform.

## 1. Concurrency & Race Conditions (Double Bookings)

### Bug 1.1: The "Check-Then-Act" (SELECT then UPDATE) Race Condition

CRITICAL

#### THE FLAWED PATTERN

Querying availability with `SELECT status FROM slots WHERE id = ?`, and if available, running `UPDATE slots SET status = 'held'`.

**Catastrophic Consequence:** When two players click the same 7:00 PM slot within the same second, both read `'available'` simultaneously. Both proceed to pay, resulting in two paying groups arriving at the physical venue at the same time.

#### THE PRODUCTION FIX

Eliminate the read step. Use an **Atomic Conditional Update** that guarantees single-winner execution:

```
-- Atomic conditional state transition
UPDATE slots
SET status = 'held', held_until = $1, held_by_user_id = $2
WHERE id = $3 AND status = 'available';
```

Check `result.rowCount === 1`. If 0 rows were affected, another player acquired the lock milliseconds earlier. Immediately abort and return a `409 Conflict`.

### Bug 1.2: The Connection Pool "Fake Transaction" Trap

CRITICAL

#### THE FLAWED PATTERN

Executing `await pool.query('BEGIN')`, followed by separate `pool.query(...)` calls, and ending with `await pool.query('COMMIT')`.

> **Silent Failure:** In connection pools (such as `pg.Pool`), each `pool.query()` fetches an arbitrary connection from the pool and immediately releases it. `BEGIN` runs on Connection A, the `INSERT` runs on Connection B (non-transactional), and `COMMIT` runs on Connection C. **There is zero transaction isolation.**

#### THE PRODUCTION FIX

Acquire a dedicated, pinned client from the pool for the lifetime of the transaction:

```
const client = await pool.connect();
try {
  await client.query('BEGIN');
  // Execute atomic slot status change + booking record creation
  await client.query('COMMIT');
} catch (err) {
  await client.query('ROLLBACK');
  throw err;
} finally {
  client.release(); // Return connection to pool
}
```

### Bug 1.3: Missing Database-Level Unique & Partial Constraints

HIGH

#### THE FLAWED PATTERN

Relying purely on JavaScript application code to prevent duplicate slots or duplicate bookings without database-level schema constraints.

> **Consequence:** When under high load, unexpected memory limits or race conditions cause duplicate slots or multiple confirmed bookings pointing to the same slot.

#### THE PRODUCTION FIX

Enforce constraints directly on the database engine:

```
-- Prevent duplicate slot intervals on the same day
ALTER TABLE slots ADD CONSTRAINT unique_slot_date_start UNIQUE (date, start_time);

-- Partial Unique Index: only 1 active booking per slot (ignoring cancelled bookings)
CREATE UNIQUE INDEX idx_single_confirmed_booking_per_slot
ON bookings (slot_id)
WHERE booking_status IN ('confirmed', 'completed');
```

### Bug 1.4: Concurrent On-Demand Slot Generation Spikes

MEDIUM

#### THE FLAWED PATTERN

Two users open an unseeded date simultaneously. Both call `ensureSlotsForDate(date)`. Both check `COUNT(*) == 0` and proceed to insert slots.

> **Consequence:** 48 slots appear on the calendar instead of 24. Users see two 6:00 PM slots and book different instances of the same hour.

#### THE PRODUCTION FIX

Use `INSERT ... ON CONFLICT (date, start_time) DO NOTHING` or an application-level mutex / Redis distributed lock.

## 2. Payment Gateway vs. Slot Hold Expiry (Inventory Desync)

### Bug 2.1: The Asymmetric Hold Expiry Window

CRITICAL

#### THE FLAWED PATTERN

The backend holds a slot for 10 minutes, but the payment gateway session (Cashfree, Razorpay, Stripe) remains valid for 15 to 30 minutes.

> **Catastrophic Consequence:** Player A initiates checkout at 12:00. At 12:10, Player A's hold expires and the slot returns to `available`. Player B holds and pays for the slot at 12:11. At 12:12, Player A completes UPI entry on their open payment tab. Money is debited from Player A, but the slot is already owned by Player B.

#### THE PRODUCTION FIX

Pass explicit order expiration parameters to the payment gateway matching the exact slot hold duration:

```
const holdExpiryIso = new Date(Date.now() + 10 * 60 * 1000).toISOString();
const gatewayOrderPayload = {
  order_id: orderId,
  order_amount: advanceAmount,
  order_expiry_time: holdExpiryIso // Gateway rejects payments after minute 10
};
```

In `/verify`, if a payment succeeds but the slot was already reassigned, the system must **never overwrite the other player**. It must immediately issue an automated refund API call.

### Bug 2.2: Webhook vs. Frontend Redirect Race Condition

HIGH

#### THE FLAWED PATTERN

Upon successful payment, both the client browser redirect ( `POST /api/bookings/verify` ) and the server webhook ( `POST /api/webhooks` ) fire concurrently.

> **Consequence:** Duplicate confirmations, double-counting revenue in analytics, or race conditions attempting to mark the slot booked twice.

#### THE PRODUCTION FIX

Implement idempotency based on the unique gateway order ID:

```
-- Atomic conditional state confirmation
UPDATE bookings
SET booking_status = 'confirmed', payment_verified = true, advance_paid_amount = $1
WHERE gateway_order_id = $2 AND booking_status = 'pending';
```

If `rowCount === 0`, the other handler already processed the confirmation. Exit cleanly without re-executing business logic.

### Bug 2.3: Unauthenticated / Forged Webhook Attacks

CRITICAL

#### THE FLAWED PATTERN

Accepting webhook payloads at `POST /api/webhooks` without validating the gateway's cryptographic signature.

> **Vulnerability:** Anyone can send a forged POST request with `{ order_status: 'PAID' }` and book any slot on the turf for ₹0.

#### THE PRODUCTION FIX

Enforce HMAC-SHA256 signature verification on the raw request body before parsing JSON:

```
const signature = req.headers['x-webhook-signature'];
const expected = crypto
  .createHmac('sha256', process.env.GATEWAY_SECRET)
  .update(rawBody)
  .digest('base64');

if (signature !== expected) {
  return res.status(401).json({ error: 'Invalid webhook signature' });
}
```

## 3. Timezone, Midnight Rollover & Date Traps

### Bug 3.1: The UTC Midnight Off-By-One Calendar Glitch

CRITICAL

#### THE FLAWED PATTERN

Parsing date strings with `new Date("YYYY-MM-DD")` in JavaScript.

> **Subtle Bug:** JavaScript parses ISO date-only strings as **UTC Midnight** (e.g., `2026-10-15T00:00:00.000Z`). When evaluated on a device or server behind UTC (such as Americas), `getDate()` evaluates to the **14th** instead of the 15th. Users book slots for yesterday.

#### THE PRODUCTION FIX

Always parse `[year, month, day]` explicitly and construct midday local date instances:

```
const [y, m, d] = dateStr.split('-').map(Number);
const localMiddayDate = new Date(y, m - 1, d, 12, 0, 0);

// Deriving today in the venue timezone (e.g. IST)
const todayIST = new Date().toLocaleDateString('en-CA', { timeZone: 'Asia/Kolkata' });
```

### Bug 3.2: Overnight & Post-Midnight Operating Hours (18:00 to 02:00)

HIGH

#### THE FLAWED PATTERN

Generating slots using naive minute increments: `for (min = start; min + duration <= end; min += duration)`.

> **Consequence:** When a sports turf operates overnight (e.g., 18:00 to 02:00), `start (1080) > end (120)`. The loop condition is immediately false and generates **zero slots**.

#### THE PRODUCTION FIX

If `endTotalMinutes <= startTotalMinutes`, add 24 hours (1440 minutes) to the end boundary:

```
let endTotalMinutes = endHour * 60 + endMin;
if (endTotalMinutes <= startTotalMinutes) {
  endTotalMinutes += 24 * 60; // Extends past midnight
}
```

### Bug 3.3: Booking Past Slots for "Today"

MEDIUM

#### THE FLAWED PATTERN

Checking only whether `slot.date >= today` when allowing bookings.

> **Consequence:** At 7:00 PM today, users can still book the 10:00 AM slot from this morning.

#### THE PRODUCTION FIX

Enforce real-time venue clock verification during the hold transition:

```
const currentTimeVenue = new Date().toLocaleTimeString('en-GB', {
  timeZone: 'Asia/Kolkata', hour12: false
}).slice(0, 5);

if (slot.date === todayVenue && slot.start_time <= currentTimeVenue) {
  throw new Error('This time slot has already elapsed.');
}
```

## 4. Slot Seeding & Admin Dynamic Configuration

### Bug 4.1: Slot Duration Change Collisions (60 min to 90 min)

CRITICAL

THE FLAWED PATTERN

Admin updates slot duration from 60 minutes to 90 minutes. Backend deletes unbooked slots and re-seeds.

> **Catastrophic Overlap:** An existing 60-minute booking exists at 19:00–20:00. The new 90-minute generator seeds slots at 18:00–19:30 and 19:30–21:00. The new `19:30–21:00` slot directly collides with the booked `19:00–20:00` slot. Two groups arrive for the turf at 19:30.

THE PRODUCTION FIX

Implement mathematical interval overlap checks before creating any new slot:

```
-- Slot A overlaps with Slot B if and only if:
SELECT 1 FROM slots
WHERE date = $1
  AND status IN ('booked', 'held')
  AND start_time < $newEndTime
  AND end_time > $newStartTime;
```

If any overlap exists with existing bookings, the slot generator must skip that interval and flag an admin warning.

## 5. Financial Integrity & Rounding Discrepancies

### Bug 5.1: Client-Side Price Tampering

CRITICAL

THE FLAWED PATTERN

Trusting prices or advance amounts passed in the client request body (e.g. `req.body.price`).

THE PRODUCTION FIX

Never trust client prices. Server must read the authoritative slot price from the DB and recalculate advance/balance amounts.

### Bug 5.2: Advance Percentage Rounding Inconsistencies

MEDIUM

THE FLAWED PATTERN

Computing `advance = slotPrice * 0.40` and `balance = slotPrice * 0.60` independently.

> **Ledger Mismatch:** With odd amounts (e.g. ₹1255 * 0.40 = 502, ₹1255 * 0.60 = 753), independent rounding causes `advance + balance !== total`.

THE PRODUCTION FIX

Always calculate balance strictly by subtraction:

```
const advanceAmount = Math.round((totalPrice * advancePct) / 100);
const balanceAmount = totalPrice - advanceAmount; // Guarantees sum === total
```

## 6. Cancellation & Rescheduling Edge Cases

### Bug 6.1: Client-Side Cancellation Window Bypasses

HIGH

**THE TRAP & FIX**

The UI hides the cancel button 4 hours prior, but the API endpoint lacks verification. Always evaluate the cutoff server-side:

```
const slotStartUtc = new Date(`${booking.date}T${booking.start_time}:00+05:30`).getTime();
const cutoffLimit = slotStartUtc - (cancellationHours * 60 * 60 * 1000);
if (Date.now() > cutoffLimit) {
  return res.status(400).json({ error: 'Cancellation deadline has expired.' });
}
```

### Bug 6.2: The Ghost Slot on Cancellation

MEDIUM

**THE TRAP & FIX**

Marking a booking as `'cancelled'` without atomically releasing the slot back to `'available'`. Ensure an atomic multi-table transaction updates both records.

## 7. Serverless & Background Worker Pitfalls

### Bug 7.1: The Frozen `setInterval` on Serverless / PaaS

HIGH

THE FLAWED PATTERN

Relying on an in-memory `setInterval(revertExpiredHolds, 60000)` on Vercel, AWS Lambda, or auto-sleeping PaaS containers.

> **Silent Failure:** Serverless runtimes pause execution between HTTP requests. Timers do not fire! Expired slots remain locked indefinitely until manual intervention.

THE PRODUCTION FIX

Combine **Lazy Evaluation on Read** with an external scheduled cron:

```
// In GET /api/slots: Revert expired holds lazily before querying
app.get('/api/slots', async (req, res) => {
  await revertExpiredHolds(); // Self-healing on every customer request
  // proceed to return fresh slots
});
```

## 8. Master Architecture Production Blueprint

| SUBSYSTEM | VULNERABILITY / COMMON BUG | MANDATORY PRODUCTION DEFENSE |
|---|---|---|
| **Concurrency** | Two users book the same slot simultaneously. | Atomic conditional `UPDATE ... WHERE status = 'available'` and DB-level unique indexes. |
| **DB Transactions** | Broken isolation due to connection pool query execution. | Always pin transactions to a dedicated client ( `await pool.connect()` ). |
| **Payment Session** | User pays after slot hold timeout; slot is already taken. | Sync gateway `order_expiry_time` with slot hold window; auto-refund fallback if taken. |
| **Webhooks** | Forged payment success webhooks; duplicate processing. | Mandatory HMAC-SHA256 signature verification and order ID idempotency locks. |
| **Timezones** | UTC midnight off-by-one shifts; calendar showing wrong day. | Midday local constructors `[Y, M, D]` and strict venue timezone resolution ( `en-CA` ). |
| **Operating Hours** | Loops failing on cross-midnight schedules (18:00 to 02:00). | Modulo 24 calculations (add 1440 min if `end < start` ). |
| **Admin Changes** | Changing slot duration from 60m to 90m creates slot collisions. | Mathematical interval overlap validation ( `startA < endB AND endA > startB` ). |
| **Cancellations** | Bypassing 4-hour cancellation rule; orphaned locked slots. | Server-side venue clock verification; atomic rollback of slot to `available` . |
| **Serverless Holds** | In-memory timers freezing when container sleeps. | Lazy hold expiry check on every slot read + external cron ping. |
| **Frontend State** | Stale slot grid; rapid double clicks on checkout button. | Disable button on submit with idempotency key; poll or SSE for live slot status. |